

Code main.c:

```
/* Includes -----
-----*/
#include "main.h"

/* Private includes -----
-----*/
/* USER CODE BEGIN Includes */
#include "stdio.h"
#include "string.h"
#include "ctype.h"
/* USER CODE END Includes */

/* Private variables -----
-----*/
ADC_HandleTypeDef hadc1;
TIM_HandleTypeDef htim2;
UART_HandleTypeDef huart1;

/* USER CODE BEGIN PV */
uint8_t rx_char; (**1a)
char tx_data[20] = "Hello\n\r";
char nhiet_do[20] = "Nhiet do cao\n\r";
uint8_t led_state[8] = {0}; (**1b)
uint32_t adc_val = 0;
uint8_t system_running = 0; (*4) (**8)
/* USER CODE END PV */

/* Private function prototypes -----
-----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART1_UART_Init(void);
static void MX_TIM2_Init(void);
static void MX_ADC1_Init(void);
/* USER CODE BEGIN PFP */
float Read_Temperature(void);
/* USER CODE END PFP */

/* USER CODE BEGIN 0 */
#define LCD_RS_Pin (***7)
GPIOA,GPIO_PIN_14
#define LCD_E_Pin
GPIOA,GPIO_PIN_13
#define LCD_D4_Pin
GPIOA,GPIO_PIN_6
#define LCD_D5_Pin
GPIOA,GPIO_PIN_5
#define LCD_D6_Pin
GPIOA,GPIO_PIN_4
#define LCD_D7_Pin
GPIOA,GPIO_PIN_3
#define LCD_Set_Pin(LCD_Pin)
HAL_GPIO_WritePin(LCD_Pin,GPIO
_PIN_SET)

* @brief System Clock Configuration
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct =
    {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct =
    {0};
    RCC_PeriphCLKInitTypeDef PeriphClkInit =
    {0};

    /** Initializes the RCC Oscillators according to
    the specified parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType =
    RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSIState =
    RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue =
    RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState =
    RCC_PLL_NONE;
    if
    (HAL_RCC_OscConfig(&RCC_OscInitStruct)
    != HAL_OK)
    {
        Error_Handler();
    }
    /** Initializes the CPU, AHB and APB buses
    clocks
    */
    RCC_ClkInitStruct.ClockType =
    RCC_CLOCKTYPE_HCLK|RCC_CLOCKTY
    PE_SYSCLK

[RCC_CLOCKTYPE_PCLK1|RCC_CLOCKT
    YPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource =
    RCC_SYSCLKSOURCE_HSI;
    RCC_ClkInitStruct.AHBCLKDivider =
    RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider =
    RCC_HCLK_DIV1;
    RCC_ClkInitStruct.APB2CLKDivider =
    RCC_HCLK_DIV1;

    if
    (HAL_RCC_ClockConfig(&RCC_ClkInitStruct
    , FLASH_LATENCY_0) != HAL_OK)
    {
        Error_Handler();
    }
    PeriphClkInit.PeriphClockSelection =
    RCC_PERIPHCLK_ADC;
```

<pre> #define LCD_Reset_Pin(LCD_Pin) HAL_GPIO_WritePin(LCD_Pin,GPIO _PIN_RESET) #define LCD_Write_Pin(LCD_Pin,value) HAL_GPIO_WritePin(LCD_Pin,value) #define LCD_Delay_ms(time) HAL_Delay(time) #define LCD_GetBit(value,bit) ((value>>bit) & 1) #define LCD_Send_cmd(Value) LCD_Write_8bit(Value,0) #define LCD_Send_cmd_4bit(Value) LCD_Write_4bit(Value,0) #define LCD_Send_data(Value) LCD_Write_8bit(Value,1) #define LCD_SetCursor(row,col) LCD_Send_cmd(0x80+(row*64)+col) #define LCD_clear() LCD_Write_8bit(0x01,0) (****7) </pre>	<pre> PeriphClkInit.AdcClockSelection = RCC_ADCCLK2_DIV2; if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkI nit) != HAL_OK) { Error_Handler(); } } </pre>
<pre> void LCD_Write_4bit(uint8_t value, uint8_t cmd_data) {(****8) LCD_Write_Pin(LCD_RS_Pin, cmd_data); LCD_Set_Pin(LCD_E_Pin); LCD_Delay_ms(1); LCD_Write_Pin(LCD_D4_Pin, LCD_GetBit(value,0)); LCD_Write_Pin(LCD_D5_Pin, LCD_GetBit(value,1)); LCD_Write_Pin(LCD_D6_Pin, LCD_GetBit(value,2)); LCD_Write_Pin(LCD_D7_Pin, LCD_GetBit(value,3)); LCD_Delay_ms(1); LCD_Reset_Pin(LCD_E_Pin); LCD_Delay_ms(1); } void LCD_Write_8bit(uint8_t value, uint8_t cmd_data) {(****8) LCD_Write_4bit((value >> 4) & 0x0F, cmd_data); LCD_Write_4bit(value & 0x0F, cmd_data); } void LCD_Send_string(char *ptr) {(****10) int i; int len; len = strlen(ptr); for (i = 0; i < len; i++) { LCD_Write_8bit(ptr[i], 1); } } void LCD_Send_number(float number){(****10) char buffer[8]; </pre>	<pre> * @brief ADC1 Initialization Function static void MX_ADC1_Init(void) {(****2) /* USER CODE BEGIN ADC1_Init 0 */ /* USER CODE END ADC1_Init 0 */ ADC_ChannelConfTypeDef sConfig = {0}; /* USER CODE BEGIN ADC1_Init 1 */ /* USER CODE END ADC1_Init 1 */ /** Common config */ hadc1.Instance = ADC1; hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE; hadc1.Init.ContinuousConvMode = ENABLE; hadc1.Init.DiscontinuousConvMode = DISABLE; hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START; hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT; hadc1.Init.NbrOfConversion = 1; if (HAL_ADC_Init(&hadc1) != HAL_OK) { Error_Handler(); } /** Configure Regular Channel */ sConfig.Channel = ADC_CHANNEL_2; sConfig.Rank = ADC_REGULAR_RANK_1; sConfig.SamplingTime = ADC_SAMPLETIME_1CYCLE_5; </pre>

```
sprintf(buffer, "%.2f", number);
LCD_Send_string(buffer);
}
```

```
void LCD_init() {(***9)
LCD_Send_cmd_4bit(0x03);
LCD_Delay_ms(5);
LCD_Send_cmd_4bit(0x03);
LCD_Delay_ms(1);
LCD_Send_cmd_4bit(0x03);
LCD_Delay_ms(1);
LCD_Send_cmd_4bit(0x02);
LCD_Delay_ms(1);
LCD_Send_cmd(0x28);
LCD_Delay_ms(1);
LCD_Send_cmd(0x0C);
LCD_Delay_ms(1);
}
```

```
void UART1_SendString(char
*str){(**7)(***6)
HAL_UART_Transmit(&huart1, (uint8_t*)str,
strlen(str), 1000);
}
```

```
void StartSystem(void)(*3a) (**9a) (***12)
{
// Bắt đầu hệ thống
UART1_SendString("System started\r\n");
system_running = 1;
UART1_SendString(tx_data);
HAL_UART_Receive_IT(&huart1,
&rx_char, 1); (**6b)
}
```

```
void StopSystem(void) (*3b) (**9b) (***12)
{
// Dừng hệ thống
UART1_SendString("System stopped\r\n");
system_running = 0;
}
```

```
void led_control(uint8_t pin, uint8_t state,
uint8_t index) {(**2)
if (state == 1) {
HAL_GPIO_WritePin(GPIOB, pin,
GPIO_PIN_SET);
led_state[index] = state;
} else {
HAL_GPIO_WritePin(GPIOB, pin,
GPIO_PIN_RESET);
for(int i = 0; i < index; i++){
led_state[i] = state;
}
}
```

```
if (HAL_ADC_ConfigChannel(&hadc1,
&sConfig) != HAL_OK)
{
Error_Handler();
}
/* USER CODE BEGIN ADC1_Init 2 */

/* USER CODE END ADC1_Init 2 */

}
```

```
/**
 * @brief TIM2 Initialization Function
static void MX_TIM2_Init(void)
(**2)

/* USER CODE BEGIN TIM2_Init 0 */

/* USER CODE END TIM2_Init 0 */

TIM_ClockConfigTypeDef
sClockSourceConfig = {0};
TIM_MasterConfigTypeDef sMasterConfig =
{0};
```

```
/* USER CODE BEGIN TIM2_Init 1 */

/* USER CODE END TIM2_Init 1 */
htim2.Instance = TIM2;
htim2.Init.Prescaler = 8000;
htim2.Init.CounterMode =
TIM_COUNTERMODE_UP;
htim2.Init.Period = 499;
htim2.Init.ClockDivision =
TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload =
TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim2) !=
HAL_OK)
{
Error_Handler();
}
sClockSourceConfig.ClockSource =
TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim2,
&sClockSourceConfig) != HAL_OK)
{
Error_Handler();
}
sMasterConfig.MasterOutputTrigger =
TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode =
TIM_MASTERSLAVEMODE_DISABLE;
if
(HAL_TIMEx_MasterConfigSynchronization(
&htim2, &sMasterConfig) != HAL_OK)
```

```

}

void send_led_state(void) (**3)
{
    UART1_SendString("\r\nState_LED: ");
    uint8_t led_state_str[9];
    for (int i = 0; i < 8; i++)
    {
        led_state_str[i] = led_state[i] + '0';
    }
    led_state_str[8] = '\r';
    HAL_UART_Transmit(&huart1,
led_state_str, 9, 1000);
}

void send_temperature(void)
{
    float temp = Read_Temperature();
    char temp_msg[30];
    sprintf(temp_msg, sizeof(temp_msg),
"\r\nNhiet do: %.2f oC\r\n", temp);
    HAL_UART_Transmit(&huart1,
(uint8_t*)temp_msg, strlen(temp_msg),
HAL_MAX_DELAY);
}

int main(void) (****4)
{
    /* Reset of all peripherals, Initializes the Flash
interface and the Systick. */
    HAL_Init();

    /* Configure the system clock */
    SystemClock_Config();

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_USART1_UART_Init();
    MX_TIM2_Init();
    MX_ADC1_Init();

    /* USER CODE BEGIN 2 */
    LCD_init();
    char msg[30];
    HAL_TIM_Base_Start_IT(&htim2); (**3)

    /* USER CODE END 2 */
    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1) (*5)
    {
        /* USER CODE END WHILE */
        /* USER CODE BEGIN 3 */
        if(system_running != 0){
            static float previous_temp = -1000.0;
            float temp = Read_Temperature();
        }
    }

    {
        Error_Handler();
    }
    /* USER CODE BEGIN TIM2_Init 2 */

    /* USER CODE END TIM2_Init 2 */

}

    * @brief USART1 Initialization Function
    static void MX_USART1_UART_Init(void)
    {(****3)

    /* USER CODE BEGIN USART1_Init 0 */

    /* USER CODE END USART1_Init 0 */

    /* USER CODE BEGIN USART1_Init 1 */

    /* USER CODE END USART1_Init 1 */
    huart1.Instance = USART1;
    huart1.Init.BaudRate = 9600;
    huart1.Init.WordLength =
UART_WORDLENGTH_8B;
    huart1.Init.StopBits = UART_STOPBITS_1;
    huart1.Init.Parity = UART_PARITY_NONE;
    huart1.Init.Mode = UART_MODE_TX_RX;
    huart1.Init.HwFlowCtl =
UART_HWCONTROL_NONE;
    huart1.Init.OverSampling =
UART_OVERSAMPLING_16;
    if (HAL_UART_Init(&huart1) != HAL_OK)
    {
        Error_Handler();
    }
    /* USER CODE BEGIN USART1_Init 2 */
    HAL_UART_Receive_IT(&huart1, &rx_char,
1); (**6a)
    /* USER CODE END USART1_Init 2 */

}

static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOA_CLK_ENABLE();(*1)
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA,
GPIO_PIN_3|GPIO_PIN_4|GPIO_PIN_5|GPIO
_PIN_6

|GPIO_PIN_11|GPIO_PIN_13|GPIO_PIN_14,
GPIO_PIN_RESET);

```

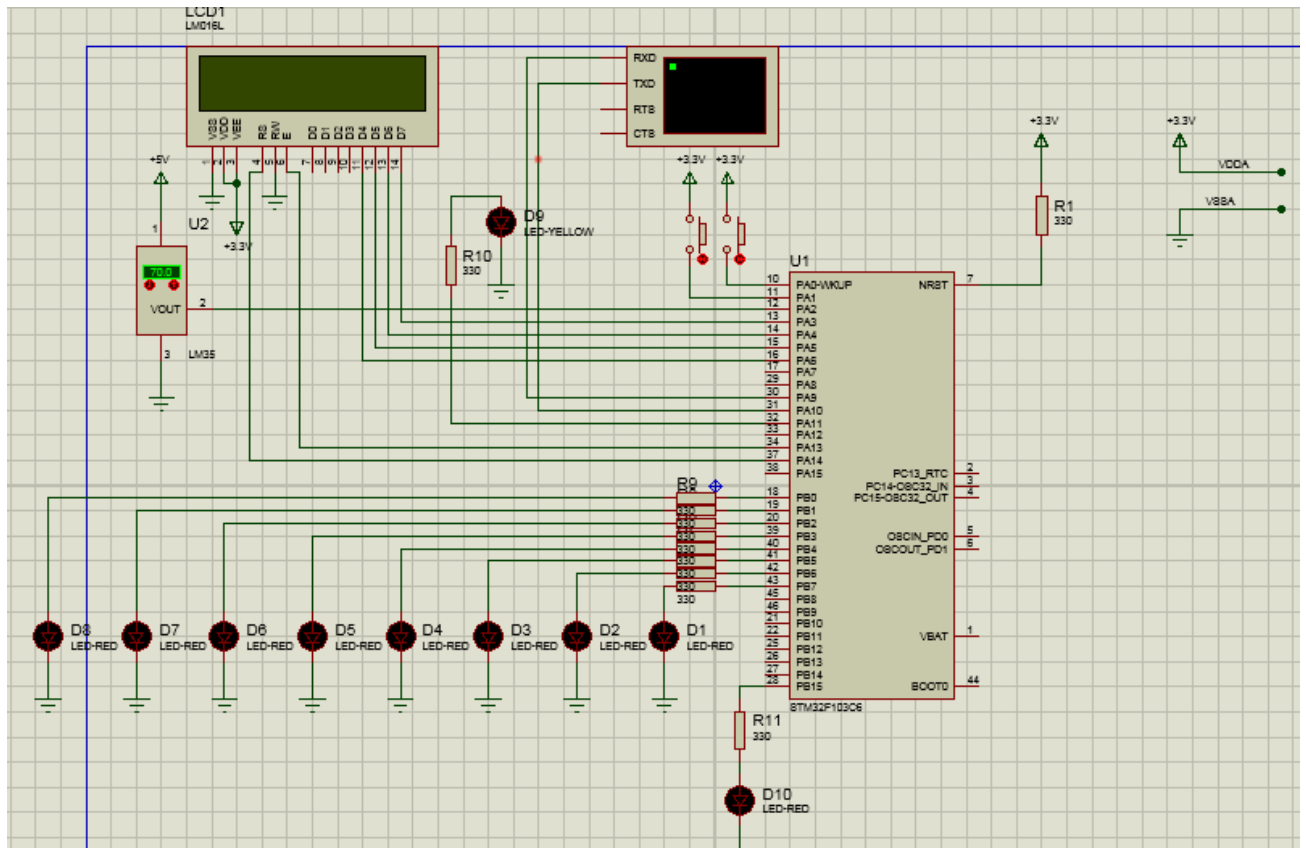
<pre> if(temp != previous_temp){ if(temp < 70){ snprintf(msg, sizeof(msg), "Nhiet do: %.2f\r\n", temp); HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY); } else{ UART1_SendString(nhiet_do); } previous_temp = temp; LCD_clear(); LCD_Send_string("Nhiet do: "); if(temp >= 70){ LCD_Send_string("Cao"); HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, 1); } else{ HAL_GPIO_WritePin(GPIOB, GPIO_PIN_15, 0); LCD_Send_number(temp); } HAL_Delay(500); } } /* USER CODE END 3 */ } void Error_Handler(void) { /* USER CODE BEGIN Error_Handler_Debug */ /* User can add his own implementation to report the HAL error return state */ __disable_irq(); while (1) { } /* USER CODE END Error_Handler_Debug */ } </pre>	<pre> /*Configure GPIO pin Output Level */ HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0 GPIO_PIN_1 GPIO_PIN_2 GPIO _PIN_15 GPIO_PIN_3 GPIO_PIN_4 GPIO_PIN_5 GPIO _PIN_6 GPIO_PIN_7, GPIO_PIN_RESET); /*Configure GPIO pins : PA0 PA1 */(*1) GPIO_InitStruct.Pin = GPIO_PIN_0 GPIO_PIN_1; GPIO_InitStruct.Mode = GPIO_MODE_IT_RISING; GPIO_InitStruct.Pull = GPIO_PULLDOWN; HAL_GPIO_Init(GPIOA, &GPIO_InitStruct); /*Configure GPIO pins : PA3 PA4 PA5 PA6 PA11 PA13 PA14 */ (**1) GPIO_InitStruct.Pin = GPIO_PIN_3 GPIO_PIN_4 GPIO_PIN_5 GPIO _PIN_6 GPIO_PIN_11 GPIO_PIN_13 GPIO_PIN_14; GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; GPIO_InitStruct.Pull = GPIO_NOPULL; GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW; HAL_GPIO_Init(GPIOA, &GPIO_InitStruct); /*Configure GPIO pins : PB0 PB1 PB2 PB15 PB3 PB4 PB5 PB6 PB7 */ (**5) GPIO_InitStruct.Pin = GPIO_PIN_0 GPIO_PIN_1 GPIO_PIN_2 GPIO _PIN_15 GPIO_PIN_3 GPIO_PIN_4 GPIO_PIN_5 GPIO _PIN_6 GPIO_PIN_7; GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; GPIO_InitStruct.Pull = GPIO_NOPULL; GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW; HAL_GPIO_Init(GPIOB, &GPIO_InitStruct); /* EXTI interrupt init*/(*1) HAL_NVIC_SetPriority(EXTI0_IRQn, 0, 0); HAL_NVIC_EnableIRQ(EXTI0_IRQn); HAL_NVIC_SetPriority(EXTI1_IRQn, 0, 0); </pre>
--	---

	<pre> HAL_NVIC_EnableIRQ(EXTI1_IRQn); } /* USER CODE BEGIN 4 */ float Read_Temperature(void){(****1) HAL_ADC_Start(&hadc1); HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY); adc_val = HAL_ADC_GetValue(&hadc1); HAL_ADC_Stop(&hadc1); float voltage = (adc_val * 5.0) / 4096.0; float temperature = (voltage - 0.002) * 100.0; return temperature; } /* USER CODE END 4 */ </pre>
--	---

Code trong stm32fxx_it.c:

<pre> /* Includes ----- -----*/ #include "main.h" #include "stm32f1xx_it.h" /* USER CODE BEGIN PV */ extern uint8_t rx_char; extern char tx_data[20]; extern uint8_t led_state[8]; extern uint8_t system_running; uint8_t save = 0; /* USER CODE END PV */ /* USER CODE BEGIN PFP */ (*7) void led_control(uint8_t pin, uint8_t state, uint8_t index); void StartSystem(void); void StopSystem(void); void send_led_state(void); void send_temperature(void); /* USER CODE END PFP */ /* External variables ----- -----*/ extern TIM_HandleTypeDef htim2; extern UART_HandleTypeDef huart1; * @brief This function handles EXTI line0 interrupt. (*2a)(****11a) void EXTI0_IRQHandler(void) { /* USER CODE BEGIN EXTI0_IRQn 0 */ if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET) { // Bắt đầu hệ thống </pre>	<pre> * @brief This function handles USART1 global interrupt. void USART1_IRQHandler(void) (*6) (**4) (**10) { /* USER CODE BEGIN USART1_IRQn 0 */ /* USER CODE END USART1_IRQn 0 */ HAL_UART_IRQHandler(&huart1); /* USER CODE BEGIN USART1_IRQn 1 */ if(system_running != 0){ HAL_UART_Transmit(&huart1, &rx_char, 1, 100); if (HAL_UART_Receive_IT(&huart1, &rx_char, 1) == HAL_OK) { // Phân tích dữ liệu nhận được if (rx_char == '1') { led_control(GPIO_PIN_0, 1, 0); } else if (rx_char == '2') { led_control(GPIO_PIN_1, 1, 1); } else if (rx_char == '3') { led_control(GPIO_PIN_2, 1, 2); } else if (rx_char == '4') { led_control(GPIO_PIN_3, 1, 3); } else if (rx_char == '5') { led_control(GPIO_PIN_4, 1, 4); } else if (rx_char == '6') { led_control(GPIO_PIN_5, 1, 5); } else if (rx_char == '7') { led_control(GPIO_PIN_6, 1, 6); } else if (rx_char == '8') { led_control(GPIO_PIN_7, 1, 7); } else if (rx_char == '0') { // Tắt tất cả đèn LED led_control(GPIO_PIN_0 GPIO_PIN_1 GPIO_PIN_2 GPIO_PIN_3 </pre>
---	---

<pre> StartSystem(); } /* USER CODE END EXTI0_IRQn 0 */ HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0); /* USER CODE BEGIN EXTI0_IRQn 1 */ /* USER CODE END EXTI0_IRQn 1 */ } * @brief This function handles EXTI line1 interrupt. (*2b) (****11b) void EXTI1_IRQHandler(void) { /* USER CODE BEGIN EXTI1_IRQn 0 */ if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == GPIO_PIN_SET) { // Dừng hệ thống StopSystem(); } /* USER CODE END EXTI1_IRQn 0 */ HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_1); /* USER CODE BEGIN EXTI1_IRQn 1 */ /* USER CODE END EXTI1_IRQn 1 */ } * @brief This function handles TIM2 global interrupt. void TIM2_IRQHandler(void) (****4) { /* USER CODE BEGIN TIM2_IRQn 0 */ /* USER CODE END TIM2_IRQn 0 */ HAL_TIM_IRQHandler(&htim2); /* USER CODE BEGIN TIM2_IRQn 1 */ HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_11); if(save != 0){ HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_0); } /* USER CODE END TIM2_IRQn 1 */ } </pre>	<pre> GPIO_PIN_4 GPIO_PIN_5 GPIO_PIN_6 GPIO_PIN_7, 0, 8); } if (rx_char == 'S') { send_led_state(); } if (rx_char == 'T') { send_temperature(); } if (rx_char == 'A') { save = 1; } if (rx_char == 'O') { save = 0; HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0 GPIO_PIN_1 GPIO_PIN_2 GPIO_PIN_3 GPIO_PIN_4 GPIO_PIN_5 GPIO_PIN_6 GPIO_PIN_7, 0); } } /* USER CODE END USART1_IRQn 1 */ } </pre>
--	--



Đề:

1 nút nhấn Start để bắt đầu và 1 nút Stop để dừng hệ thống, sử dụng ngắt ngoài.(*)

+ 1) **Cấu hình GPIO** và ngắt ngoài cho PA0 và PA1: hàm **MX_GPIO_Init** trong file **main.c** (Kích hoạt clock cho cổng GPIOA; Cấu hình chân PA0 và PA1 làm ngõ vào với chế độ ngắt ngoài trên sườn lên và sử dụng Pull-down resistor; Kích hoạt ngắt ngoài cho EXTI dòng 0 và 1 tương ứng với PA0 và PA1). (2)

+ 2) Hàm xử lý ngắt cho PA0 (Bắt đầu hệ thống): hàm **EXTI0_IRQHandler** trong file **stm32f1xx_it.c** (Khi có ngắt xảy ra trên chân PA0, hàm EXTI0_IRQHandler sẽ được gọi; Kiểm tra trạng thái chân PA0, nếu đang ở mức SET (nút được nhấn), gọi hàm **StartSystem()** để bắt đầu hệ thống; Gọi hàm **HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0)** để xóa cờ ngắt và chuẩn bị cho lần ngắt tiếp theo); Tương tự với hàm **EXTI1_IRQHandler** (hàm **StopSystem()** gọi để dừng hệ thống). 2

+ 3) Hàm **StartSystem()** và **StopSystem()** trong file **main.c**: (**StartSystem()**: Gửi thông báo bắt đầu hệ thống và lời chào qua UART, đặt biến **system_running** thành 1 để biểu thị hệ thống đang chạy và bắt đầu nhận dữ liệu UART không đồng bộ; **StopSystem()**: Gửi thông báo dừng hệ thống và đặt **system_running** về 0 để biểu thị hệ thống đã dừng). (2)

+ 4) **Biến toàn cục điều khiển** trạng thái hệ thống trong file **main.c**: (Biến này được sử dụng để kiểm soát việc hệ thống có đang hoạt động hay không. Khi **system_running** bằng 1, hệ thống sẽ thực hiện các chức năng chính trong vòng lặp **main(void)** và hàm **USART1_IRQHandler(void)** (**stm32f**) cho phép hệ thống nhận và xử lý dữ liệu UART để thực hiện các hành động dựa trên ký tự nhận được. (1)

+ 5) Sử dụng biến **system_running** trong **vòng lặp chính** trong hàm **main()** trong file **main.c**: (Vòng lặp vô hạn **while(1)** kiểm tra trạng thái của **system_running**; Nếu **system_running** khác 0 (hệ thống đang chạy), thì thực hiện các tác vụ trong chương trình, ví dụ như đọc nhiệt độ từ LM35, hiển thị nhiệt độ lên LCD, xử lý UART,...(1)

+ **6)** Khi có **ngắt nhận UART** xảy ra do biến `system_running = 1` và tức là khi nhận được một ký tự mới qua UART), hàm **USART1_IRQHandler(void)** trong file (**stm32f1xx_it.c**) sẽ thực hiện các tác vụ trong hàm, ví dụ như bật LED tương ứng nếu `rx_char` từ '1' đến '8' (nhập vào Virtual Terminal) và ngược lại tắt tất cả các LED nếu `rx_char` là '0', nếu `rx_char` là 'S' thì sẽ gửi trạng thái hiện tại của các LED qua UART, nếu `rx_char` là 'T' thì sẽ đọc và gửi giá trị nhiệt độ hiện tại qua UART, nếu `rx_char` là 'A' thì cho phép nhấp nháy tất cả các LED còn lại(timer) trong hàm `TIM2_IRQHandler()`, ngược lại thì `rx_char` là 'O' thì sẽ tắt tất cả các LED đã cho phép và chỉ có một LED RUN nhấp nháy(timer) như ban đầu.(1)

+ **7)** Khai báo các **hàm** trong file **stm32f1xx_it.c**: (khai báo các hàm trên để có thể gọi chúng khi `system_running = 1`). (1)

+ **8)** Bước quan trọng là phải khai báo **thư viện, biến, hàm** để có thể chạy các chức năng khi Start và Stop như (7)

8 đèn LED tương ứng với 8 thiết bị được điều khiển bật/tắt từ giao diện Virtual Terminal trong Proteus qua UART.()**

+ **1) Khai báo biến** và quản lý trạng thái của các LED trong file **main.c**: (**rx_char**: Biến lưu trữ ký tự nhận được từ UART; **led_state[8]**: Mảng lưu trạng thái bật/tắt của 8 đèn LED. Mỗi phần tử trong mảng đại diện cho một đèn LED. (2)

+ **2) Hàm điều khiển LED** trong file **main.c**: (Hàm **led_control** dùng để bật một đèn LED hoặc tắt tất cả các đèn LED; Tham số **pin** xác định **chân GPIO** tương ứng với đèn LED; **state** là trạng thái bật(1) hoặc tắt (0); **Index** là chỉ số trong mảng `led_state` (0 đến 7) để cập nhật trạng thái của đèn LED tương ứng, với `Index = 8` thì không phải chỉ số trong mảng `led_state` mà là giới hạn của vòng lặp **for** để cập nhật trạng thái của tất cả đèn LED. (1)

+ **3) Hàm gửi trạng thái(state) LED** qua UART trong file **main.c**: (Hàm **send_led_state** dùng để gửi trạng thái hiện tại của 8 đèn LED qua UART với Virtual Terminal; **Trạng thái** của mỗi đèn LED (0 hoặc 1) được chuyển thành ký tự và gửi đi). (1)

+ **4) Xử lý dữ liệu nhận được từ UART để điều khiển LED** trong file **stm32f1xx_it.c** trong hàm ngắt **USART1_IRQHandler**: (Khi nhận được ký tự qua UART, chương trình kiểm tra giá trị của **rx_char**; Nếu `rx_char` là các số từ '1' đến '8', chương trình sẽ **bật đèn** LED tương ứng bằng cách gọi hàm `led_control`; Nếu `rx_char` là '0', tắt tất cả các đèn LED sẽ được **tắt**; Nếu `rx_char` là 'S', chương trình sẽ **gửi trạng thái hiện tại** của các đèn LED qua UART; Điều này cho phép điều khiển các đèn LED từ giao diện Virtual Terminal trong Proteus. (1)

+ **5) Cấu hình GPIO cho các chân LED** trong file **main.c**: (Cấu hình các chân **GPIO PB0 đến PB7** làm ngõ ra để **điều khiển 8 đèn LED**; **Chân PB15** cũng được cấu hình, có thể sử dụng cho chức năng khác (ví dụ: cảnh báo nhiệt độ cao)). (1)

+ **6) Khởi tạo UART và nhận dữ liệu không đồng bộ** trong file **main.c**: (Bắt đầu nhận dữ liệu UART theo chế độ ngắt (Interrupt), cho phép chương trình tiếp tục thực thi mà không bị chặn khi chờ dữ liệu UART; `rx_char` là biến lưu trữ ký tự nhận được). (2)

+ **7) Hàm gửi chuỗi qua UART** trong file **main.c**: (Hàm **UART1_SendString** dùng để gửi một chuỗi ký tự qua UART; Được sử dụng để **gửi thông báo hoặc trạng thái** đến Virtual Terminal). (1)

+ **8) Cấu hình trạng thái hệ thống và biến liên quan** trong file **main.c**: (Biến `system_running` được sử dụng để kiểm soát trạng thái hoạt động của hệ thống; Khi `system_running = 1`, hệ thống **cho phép** nhận và **xử lý lệnh từ UART**). (1)

+ **9) Hàm StartSystem và StopSystem** trong file **main.c**: (Hàm StartSystem và StopSystem dùng để kích hoạt và vô hiệu hoá hệ thống điều khiển thông qua UART; Khi hệ thống chạy, việc nhận và xử lý lệnh UART sẽ được thực hiện). (2)

+ **10) Cấu hình ngắt UART và nhận dữ liệu** trong file **stm32f1xx_it.c**: (Hàm ngắt **USART1_IRQHandler** được gọi khi có dữ liệu UART nhận được; Khi hệ thống đang chạy (**system_running != 0**), ký tự nhận được sẽ được xử lý để điều khiển đèn LED,...). (1)

1 đèn LED chỉ thị trạng thái "RUN", bật/tắt mỗi 0.5 giây, sử dụng ngắt Timer.(*)**

+ **1) Cấu hình GPIO PA11** làm ngõ ra để điều khiển LED “RUN” nhấp nháy bật/tắt mỗi 0.5 giây trong file **main.c**. (1)

+ **2) Cấu hình Timer(Timer2)** trong hàm **MX_TIM2_Init(void)** trong file **main.c**: (Tính toán chu kỳ 0.5 giây: Với tần số hệ thống: 8MHz, Prescaler: 8000 (chia tần số xuống), Period: 499 (tạo chu kỳ 0.5s), Thời gian = (Prescaler + 1) x (Period + 1) / 8MHz = 0.5s). (1)

+ **3) Khởi động Timer** (Bật ngắt Timer2) trong hàm **main(void)** trong file **main.c**: (Timer2 sẽ đếm từ 0 đến 499; Khi đếm đủ thì sẽ tạo ngắt; Trong ngắt, LED được đảo trạng thái; LED sẽ đổi trạng thái mỗi 0.5 giây(tương đương nhấp nháy 1Hz)). (1)

+ **4) Xử lý trong ngắt Timer** trong hàm **TIM2_IRQHandler(void)** trong file **stm32f1xx_it.c**: (Hàm **HAL_GPIO_TogglePin** tự động đảo trạng thái của LED “RUN” mỗi khi có ngắt Timer; Không làm ảnh hưởng tới luồng chính của chương trình). (1)

Cảm biến nhiệt độ LM35 đọc nhiệt độ môi trường. Gửi dữ liệu nhiệt độ đọc từ LM35 qua UART lên máy tính, với chu kỳ 0.5 giây/lần. Hiển thị giá trị nhiệt độ đọc lên màn hình LCD và Virtual Terminal.(**)**

+ **1) Hàm Read_Temperature()** (Đọc nhiệt độ từ cảm biến LM35) trong file **main.c**: (**Khởi động ADC**: hàm **HAL_ADC_Start(&hadc1)** bắt đầu quá trình chuyển đổi analog sang digital; **Chờ hoàn thành chuyển đổi**: **HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY)** đợi cho đến khi chuyển đổi hoàn tất; **Lấy giá trị ADC**: **adc_val = HAL_ADC_GetValue(&hadc1)** đọc giá trị chuyển đổi từ ADC; **Dừng ADC**: **HAL_ADC_Stop(&hadc1)** kết thúc quá trình chuyển đổi; **Tính điện áp và nhiệt độ**). (1)

Bổ sung: Hàm **Read_Temperature()** có chức năng: Khởi động ADC để bắt đầu quá trình chuyển đổi analog sang digital; Chờ quá trình chuyển đổi hoàn tất; Lấy giá trị ADC và dừng ADC; Tính toán điện áp từ giá trị ADC đọc được và chuyển đổi điện áp thành nhiệt độ (LM35 xuất ra điện áp tỷ lệ tuyến tính với nhiệt độ(10mV/độ C); ADC đọc giá trị điện áp này và chuyển đổi thành số(từ 0 đến 4095 với độ phân giải 12-bit; Điện áp được tính bằng cách nhân giá trị ADC với (5V / 4096) và tính ra được nhiệt độ từ điện áp).

+ **2) Hàm MX_ADC1_Init(void)** trong file **main.c**: Khởi tạo và cấu hình bộ chuyển đổi (ADC) cho ADC1 trên vi điều khiển (Bật đồng hồ cho ADC1; Cấu hình các tham số cho ADC; Cấu hình kênh ADC; Kích hoạt ADC). (1)

+ **3) Khởi tạo UART** trong hàm **MX_USART1_UART_Init()** trong file **main.c**: (Cài đặt UART1 với tốc độ baud 9600, 8-bit dữ liệu, 1 stop bit, không parity, chế độ truyền và nhận).(1)

+ **4) Gửi dữ liệu trong vòng lặp chính main()** trong file **main.c**: Kiểm tra xem hệ thống có đang chạy không (**system_running != 0**), (nếu có) **Gửi dữ liệu nhiệt độ** đọc được(Nếu nhiệt độ thay đổi so với lần đọc trước, tiến hành gửi dữ liệu) hiển thị giá trị lên LCD + bật hoặc tắt LED **GPIOB_PIN_15** dựa trên nhiệt độ và qua UART hiển thị giá trị lên Virtual Terminal với chu kỳ 0.5 giây/lần (Sử dụng

HAL_Delay(500) để tạo độ trễ 500ms trong vòng lặp chính) thì xoá màn hình LCD và lập lại quá trình). (1)

+ 5) **Mảng char msg[30]** là một mảng lưu trữ ký tự; Sau khi **đọc giá trị** từ cảm biến, giá trị này được chuyển đổi thành chuỗi ký tự để dễ dàng truyền qua giao tiếp UART hoặc hiển thị trên màn hình LCD với hàm **snprintf(msg, sizeof(msg), "Nhiệt độ: %.2f\r\n", temp)**; thì **snprintf** là hàm định dạng dữ liệu nhiệt độ(temp) và lưu trữ kết quả vào mảng msg; Gửi dữ liệu qua UART: Một khi chuỗi dữ liệu đã được định dạng và lưu trữ trong msg, nó có thể được gửi đi qua UART để truyền đến máy tính hoặc các thiết bị khác qua hàm **HAL_UART_Transmit(&huart1, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY)**; hàm này gửi chuỗi msg qua UART, strlen(msg) xác định độ dài của chuỗi để biết số byte cần gửi)

+ 6) **Hàm gửi chuỗi qua UART** trong file **main.c**: (**Hàm UART1_SendString** dùng để gửi một chuỗi ký tự qua UART; Được sử dụng để **gửi thông báo hoặc trạng thái** đến Virtual Terminal). (1)

+ 7.1) Định nghĩa các **chân kết nối LCD** trong file **main.c**: (**RS**: chân này được sử dụng để chọn chế độ **gửi dữ liệu** (RS = 1) hoặc **lệnh** (RS = 0) đến LCD; **E**: chân này được sử dụng để kích hoạt quá trình **truyền dữ liệu** hoặc **lệnh** đến LCD, khi chân E được kéo xuống mức thấp sau khi đặt dữ liệu hoặc lệnh, LCD sẽ nhận và thực hiện thao tác tương ứng; **D4 – D7**: chân dữ liệu này dùng để **truyền dữ liệu 4-bit** từ vi điều khiển đến LCD. Mô hình 4-bit giúp tiết kiệm số chân GPIO cần sử dụng so với chế độ 8-bit). (1)

+ 7.2) Định nghĩa các **macro để điều khiển chân GPIO** trong file **main.c**: (**Set**: Đặt chân GPIO được chỉ định (LCD_Pin) ở trạng thái **HIGH(1)**; **Reset**: Đặt chân GPIO được chỉ định (LCD_Pin) ở trạng thái **LOW(0)**; **Write**: Ghi giá trị (value) và chân GPIO được chỉ định (**LCD_Pin**). Giá trị có thể là **GPIO_PIN_SET** hoặc **GPIO_PIN_RESET**; **Delay_ms(time)**: Tạo một khoảng thời gian trễ(time) tính bằng mili giây; **GetBit(value, bit)**: Trích xuất bit thứ bit từ giá trị value. Kết quả là bit ở vị trí bit sẽ được trả về dưới dạng 0 hoặc 1. (1)

+ 7.3) Định nghĩa các **macro để gửi lệnh và dữ liệu đến LCD** trong file **main.c**: (**Send_cmd(Value)**: Gửi một lệnh 8-bit (Value) đến LCD; **Send_cmd_4bit(Value)**: Gửi một lệnh 4-bit (Value) đến LCD; **Send_data(Value)**: Gửi dữ liệu 8-bit (Value) đến LCD để hiển thị ký tự; **SetCursor(row, col)**: Đặt vị trí con trỏ trên LCD tại hàng row và cột col (LCD 16x2 thường có địa chỉ DDRAM như sau: Hàng 0: 0x00 – 0x0F, Hàng 1: 0x40 – 0x4F) nên khi gửi lệnh (0x80 + (row * 64) + col) thì thiết lập vị trí con trỏ; **Hàm Clear()**: Xoá màn hình LCD và đặt con trỏ về vị trí đầu tiên. (1)

+ 8) **Hàm Write_4bit**: giúp **giao tiếp với màn hình LCD**, Hiển thị thông tin, nhiệt độ hoặc các dữ liệu cảm biến lên LCD bằng cách sử dụng giao thức 4-bit, số chân GPIO cần thiết được **giảm xuống**, giúp **tiết kiệm tài nguyên** trên vi điều khiển, tương tự với hàm **Write_8bit** nhưng gửi gấp đôi số bit so với **Write_4bit** giúp **tăng tốc độ truyền dữ liệu** và **đơn giản hoá mã nguồn**. (2)

+ 9) **Hàm LCD_init()**: Khởi tạo LCD ở chế độ 4-bit, thiết lập số dòng và bật màn hình.(1)

+ 10) **Hàm LCD_Send_string()** và **LCD_Send_number()** lần lượt là: Gửi chuỗi ký tự và số dạng float lên LCD. (2)

+ 11) **Xử lý ngắt và điều khiển hệ thống** (Sử dụng ngắt ngoài ở PA0 và PA1): Xử lý ngắt nút nhấn **bắt đầu** và **dừng** hệ thống trong file **stm32f1xx_it.c**, với ngắt **EXTI0** (Bắt đầu hệ thống): Khi nút nhấn kết nối với **GPIOA_PIN_0** được kích hoạt, gọi hàm **StartSystem()** để **bắt đầu** hoạt động, với ngắt **EXTI1** (dừng hệ thống): Khi nút nhấn kết nối với **GPIOA_PIN_1** được kích hoạt, gọi hàm **StopSystem()** để **dừng** hoạt động. (2)

+ **12) Hàm StartSystem():** Gửi thông báo “System started” qua UART, Đặt cờ system_running thành 1 để cho phép các hoạt động có điều kiện này hoạt động, Bắt đầu nhận dữ liệu UART không đồng bộ;
Hàm StopSystem(): Gửi thông báo “System stopped” qua UART, đặt cờ system_running thành 0 để ngưng hoạt động của toàn bộ hệ thống ngoại trừ LED “RUN”. (2)

+ **13) Khai báo thư viện, biến và hàm liên quan**

Bổ sung: Cấu hình và khởi tạo các ngoại vi cần thiết: ADC (Khởi tạo ADC1 để đọc giá trị từ cảm biến LM35; Sử dụng kênh ADC phù hợp với chân kết nối LM35), UART (Khởi tạo UART1 để giao tiếp với máy tính hoặc Virtual Terminal; Cấu hình baud rate, số bit dữ liệu; bit dừng; và chế độ hoạt động), GPIO (Cấu hình các chân GPIO cho LCD, đèn LED cảnh báo, và các nút nhấn), LCD (Khởi tạo LCD và các hàm điều khiển hiển thị. **Code khởi tạo:**

HAL_Init(); SystemClock_Config(); MX_GPIO_Init(); MX_USART1_UART_Init();	MX_TIM2_Init(); MX_ADC1_Init(); LCD_init();
---	--

Một số khai báo ví dụ: uint16_t led_pins[8] = {GPIO_PIN_5, GPIO_PIN_6, GPIO_PIN_7}; uint16_t led_pins_all = (GPIO_PIN_5 | GPIO_PIN_6 | GPIO_PIN_7); HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5, 1); HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_13) ==

Lý Thuyết

1. Các chức năng chính của CPU? ALU là gì và nó thực hiện những phép tính nào trong CPU? Control Unit có vai trò gì trong CPU? Cách nó tương tác với bộ nhớ và ALU như thế nào?
2. Phân biệt ROM và RAM? Mục đích sử dụng của ROM và RAM trong hệ thống vi xử lý?
3. Mô tả các bước mà CPU thực hiện để xử lý một lệnh từ việc lấy lệnh đến thực hiện lệnh. Các bước để đọc và ghi dữ liệu từ/ra bộ nhớ là gì?
4. Timer/Counter trong hệ thống vi xử lý? Chức năng PWM dùng để làm gì? Được tạo ra bởi khối chức năng nào? Tìm hiểu về ADC/DAC.

Câu 1: Các chức năng chính của CPU: + Lấy lệnh: CPU lấy lệnh từ bộ nhớ chính RAM thông qua bộ đếm chương trình Program Counter PC chỉ định địa chỉ lệnh tiếp theo cần thực hiện + Giải mã lệnh: Lệnh được giải mã để xác định thao tác cần thực hiện và các tài nguyên như thanh ghi hoặc địa chỉ bộ nhớ liên quan + Thực hiện lệnh: Thực hiện thao tác đã được giải mã có thể bao gồm các phép tính toán học logic hoặc thao tác với bộ nhớ + Ghi kết quả; Kết quả của phép tính hoặc thao tác được ghi lại vào thanh ghi hoặc bộ nhớ chính ALU là gì và nó thực hiện những phép tính nào trong CPU: + ALU Arithmetic Logic Unit + Định nghĩa ALU là một thành phần quan trọng của CPU chịu trách nhiệm thực hiện các phép tính toán học như cộng trừ nhân chia và các phép toán logic như AND OR NOT XOR	Câu 2: ROM (Bộ nhớ chỉ đọc) Định nghĩa: ROM là bộ nhớ chỉ đọc, nghĩa là dữ liệu được ghi vào ROM trong quá trình sản xuất và không thể thay đổi hoặc xóa dữ liệu bởi người dùng thông thường. Đặc điểm: Không biến mất khi tắt nguồn: Dữ liệu trong ROM vẫn được lưu giữ ngay cả khi nguồn điện bị ngắt. Chậm hơn so với RAM trong việc truy xuất dữ liệu. Được sử dụng để lưu trữ firmware: Chương trình cơ bản điều khiển phần cứng của hệ thống. Mục đích sử dụng: Lưu trữ firmware: Hệ điều hành cơ bản hoặc các chương trình khởi động (bootloader) cần thiết để khởi động hệ thống. Bảo mật dữ liệu: Vì dữ liệu trong ROM không thể bị thay đổi dễ dàng, nên nó được sử dụng để lưu trữ các mã bảo mật hoặc phần mềm quan trọng. Ứng dụng nhúng: Trong các thiết bị nhúng vi điều khiển, ROM chứa chương trình điều khiển thiết bị. RAM (Bộ nhớ truy cập ngẫu nhiên)
--	--

+ Phép tính thực hiện: Phép tính toán học Cộng trừ nhân chia và các phép toán số học khác
Phép toán logic AND OR NOT XOR so sánh các giá trị ví dụ bằng lớn hơn nhỏ hơn

Control Unit có vai trò gì trong CPU Cách nó tương tác với bộ nhớ và ALU như thế nào

Control Unit CU

+ Chức năng chính CU điều phối và kiểm soát các hoạt động của CPU bằng cách lấy lệnh từ bộ nhớ giải mã lệnh và phát tín hiệu điều khiển tới các thành phần khác của CPU như ALU bộ nhớ và các thiết bị ngoại vi

+ Tương tác với bộ nhớ và ALU: Tương tác với Bộ nhớ: Lấy lệnh và dữ liệu CU gửi tín hiệu đến bộ nhớ để lấy lệnh từ địa chỉ được chỉ định bởi PC và dữ liệu cần thiết cho việc thực hiện lệnh
Ghi dữ liệu Sau khi thực hiện lệnh CU điều khiển việc ghi kết quả trở lại bộ nhớ hoặc thanh ghi

+Tương tác với ALU: Phát tín hiệu điều khiển CU gửi các tín hiệu điều khiển tới ALU để thực hiện các phép tính cần thiết dựa trên lệnh đã giải mã; Quản lý dữ liệu CU đảm bảo rằng dữ liệu được chuyển đúng cách từ bộ nhớ hoặc thanh ghi vào ALU và kết quả được xử lý sau đó

Định nghĩa: RAM là bộ nhớ truy cập ngẫu nhiên, cho phép đọc và ghi dữ liệu một cách nhanh chóng và linh hoạt.

Đặc điểm: Dữ liệu biến mất khi tắt nguồn: RAM là bộ nhớ không lưu trữ lâu dài, dữ liệu sẽ bị mất khi hệ thống bị ngắt điện. Truy cập nhanh: RAM cung cấp tốc độ truy xuất dữ liệu nhanh hơn nhiều so với ROM. Có thể đọc và ghi dữ liệu: Người dùng và hệ điều hành có thể thay đổi dữ liệu trong RAM theo nhu cầu.

Mục đích sử dụng: Lưu trữ dữ liệu tạm thời: Chạy các ứng dụng và xử lý dữ liệu trong quá trình hoạt động của hệ thống. Hỗ trợ CPU: CPU sử dụng RAM để lưu trữ các dữ liệu và lệnh đang được xử lý, giúp tăng hiệu suất làm việc.

Đa nhiệm: Cho phép chạy nhiều ứng dụng cùng lúc bằng cách cung cấp không gian lưu trữ tạm thời cho từng ứng dụng. **Mục đích sử dụng trong hệ vi xử lý:**

ROM: Lưu trữ các chương trình cố định cần thiết để khởi động và vận hành cơ bản cho hệ thống vi xử lý. Bảo vệ dữ liệu quan trọng không bị thay đổi hoặc mất mát. **RAM:** Cung cấp không gian làm việc tạm thời cho hệ thống vi xử lý khi thực hiện các tác vụ và xử lý dữ liệu. Tăng hiệu suất hệ thống bằng cách giảm thời gian truy xuất dữ liệu so với việc lấy trực tiếp từ bộ nhớ lưu trữ lâu dài.

Câu 3:

a. Lấy lệnh (Fetch): Thanh ghi đếm chương trình (Program Counter - PC): CPU sử dụng PC để xác định địa chỉ bộ nhớ của lệnh tiếp theo cần thực hiện. Đọc lệnh từ bộ nhớ: CPU gửi địa chỉ từ PC đến bộ nhớ (RAM) và lấy lệnh tại địa chỉ đó. Cập nhật PC: Sau khi lấy lệnh, PC được tăng lên để trỏ đến lệnh tiếp theo trong chuỗi lệnh. **b. Giải mã lệnh (Decode):** Giải mã lệnh: Bộ giải mã (Decoder) trong CPU phân tích lệnh vừa lấy để xác định loại lệnh (ví dụ: phép tính toán học, lệnh điều khiển, lệnh truy xuất bộ nhớ) và các tham số liên quan (như thanh ghi hoặc địa chỉ bộ nhớ). Xác định nguồn và đích: CPU xác định các thanh ghi hoặc địa chỉ bộ nhớ cần thiết để thực hiện lệnh. **c. Thực hiện lệnh (Execute):** Thực hiện phép tính hoặc thao tác: Dựa trên loại lệnh, CPU thực hiện các phép toán trên ALU (Arithmetic Logic Unit) hoặc thực hiện các thao tác điều khiển khác. Truy xuất bộ nhớ (nếu cần): Nếu lệnh yêu cầu đọc hoặc ghi dữ liệu từ/ra bộ nhớ, CPU sẽ thực hiện các bước truy xuất tương ứng. **d. Ghi kết quả (Write Back):** Lưu kết quả: Kết quả của phép toán hoặc thao tác được lưu lại vào thanh ghi hoặc ghi trở lại bộ nhớ, tùy thuộc vào lệnh đã thực hiện. Cập nhật trạng thái: CPU cập nhật các cờ trạng thái (flags) để phản ánh kết

Câu 4: Timer/Counter là gì? Timer (bộ đếm thời gian) và Counter (bộ đếm sự kiện) là các khối chức năng trong vi xử lý được sử dụng để đếm thời gian hoặc số lượng sự kiện xảy ra. Chúng có thể hoạt động độc lập hoặc kết hợp tùy thuộc vào yêu cầu của ứng dụng. **Chức năng chính của Timer/Counter:** Đo thời gian: Xác định khoảng thời gian giữa các sự kiện. Tạo độ trễ: Tạo ra các khoảng thời gian trễ cố định trong chương trình. Đếm số sự kiện: Đếm số lượng các tín hiệu hoặc sự kiện xảy ra, chẳng hạn như số lần một cảm biến bị kích hoạt. Tạo tín hiệu điều khiển: Sử dụng để tạo ra các tín hiệu PWM (Pulse Width Modulation) hoặc các dạng sóng khác. Ứng dụng của Timer/Counter: Đo thời gian thực: Sử dụng trong đồng hồ, hẹn giờ, hoặc các hệ thống đòi hỏi đo thời gian chính xác. Điều khiển PWM: Tạo ra các tín hiệu PWM để điều khiển động cơ, độ sáng đèn LED, hoặc các thiết bị điện tử khác. Đồng bộ hóa: Đảm bảo các tác vụ trong hệ thống thực hiện theo đúng lịch trình.

Chức năng PWM và Khối Chức Năng Tạo PWM 1. PWM (Pulse Width Modulation) là gì? PWM là kỹ thuật điều chế độ rộng xung, trong đó độ rộng của xung (thời gian tín hiệu ở mức cao trong một chu kỳ) được điều chỉnh để kiểm soát công suất cung cấp cho tải. PWM thường được

quả của lệnh, chẳng hạn như cờ "Zero" nếu kết quả bằng 0 hoặc cờ "Carry" nếu có phép cộng gây ra số dư. **2. Các bước để đọc và ghi dữ liệu từ/ra bộ nhớ**

a. Đọc dữ liệu từ bộ nhớ (Memory Read): Xác định địa chỉ dữ liệu: CPU xác định địa chỉ bộ nhớ nơi dữ liệu cần đọc được lưu trữ. Gửi địa chỉ tới bộ nhớ: CPU gửi địa chỉ này đến bộ điều khiển bộ nhớ thông qua đường dẫn dữ liệu (bus). Bộ nhớ trả dữ liệu: Bộ nhớ (RAM) trả dữ liệu tại địa chỉ đã chỉ định về CPU. Lưu dữ liệu vào thanh ghi: Dữ liệu nhận được được lưu vào một thanh ghi trong CPU để sử dụng trong quá trình thực thi lệnh.

b. Ghi dữ liệu vào bộ nhớ (Memory Write): Xác định địa chỉ dữ liệu: CPU xác định địa chỉ bộ nhớ nơi dữ liệu cần ghi sẽ được lưu trữ. Chuẩn bị dữ liệu cần ghi: Dữ liệu cần ghi được lưu trong một thanh ghi của CPU. Gửi địa chỉ và dữ liệu tới bộ nhớ: CPU gửi cả địa chỉ và dữ liệu tới bộ điều khiển bộ nhớ thông qua đường dẫn dữ liệu (bus). Bộ nhớ lưu dữ liệu: Bộ nhớ nhận dữ liệu và lưu nó tại địa chỉ đã chỉ định.

sử dụng để điều khiển tốc độ động cơ, độ sáng đèn LED, và trong các mạch điều khiển công suất. **2. Chức năng của PWM:** Điều khiển động cơ: Điều chỉnh tốc độ quay của động cơ DC hoặc bước. Điều chỉnh độ sáng đèn LED: Thay đổi độ sáng LED bằng cách điều chỉnh tỷ lệ Duty Cycle của PWM. Điều khiển công suất: Điều chỉnh lượng công suất cung cấp đến các thiết bị điện tử khác. **3. Khối chức năng tạo PWM:** PWM thường được tạo ra bởi Timer/Counter trong vi xử lý. Timer/Counter sử dụng các thanh ghi cấu hình để thiết lập tần số và Duty Cycle của PWM. Khi được cấu hình, Timer/Counter sẽ tạo ra các xung PWM tự động mà không cần can thiệp từ CPU, giúp tiết kiệm tài nguyên và tăng hiệu suất hệ thống.

Định nghĩa: ADC là một thiết bị hoặc thành phần trong vi xử lý dùng để chuyển đổi tín hiệu analog (liên tục) thành tín hiệu số (rời rạc) để máy tính hoặc vi xử lý có thể xử lý.

Ví dụ assembly:

MOV R0, #10 ; R0 = 10 MOV R1, #5 ; R1 = 5 LSL R2, R0, #1 ; Dịch trái R0 1 bit, lưu vào R2 CMP R2, #25 ; So sánh R2 với 25 BLT less_than ; Nhảy nếu R2 < 25	ADD R3, R0, R1 ; R3 = R0 + R1 nếu R2 >= 25 B end ; Nhảy đến end less_than: SUB R3, R0, R1 ; R3 = R0 - R1 nếu R2 < 25 end: AND R4, R3, #0xF ; R4 = R3 AND 0xF; (R3 = 0b ... 0101 & 1111 = 0101 = 5)
MOV R2, #1000 ; R2 = 1000 MOV R4, #-1400 ; R4 = -1400 ADDS R5, R2, R4 ; R5 = R2 + R4 CMP R5, #100 ; So sánh R5 với 100	MOVL T R5, #10 ; Nếu R5 < 100, thì R5 = 10 MOVGT R5, #100 ; Nếu R5 > 100, thì R5 = 100

Câu hỏi: Cấu trúc bộ vi xử lý tiêu biểu lõi ARM Cortex M3 F103C8T6

1 Kiến trúc lõi ARM Cortex M3 ARMv7 M Architecture Cortex M3 dựa trên kiến trúc ARMv7 M hỗ trợ bộ tập lệnh Thumb 2 kết hợp giữa hiệu suất cao và tiết kiệm năng lượng, Bộ xử lý RISC 32 bit Xử lý các lệnh 32 bit cung cấp hiệu suất xử lý mạnh mẽ cho các ứng dụng nhúng Pipeline 3 stage Bao gồm các bước Fetch Decode và Execute giúp tăng tốc độ xử lý lệnh 2 Thành phần chính: Đơn vị số học Logic ALU; Thực hiện các phép tính số học và logic; Hỗ trợ các phép toán như cộng trừ AND OR XOR và so sánh; Đơn vị điều khiển CU, Điều phối hoạt động của toàn bộ CPU, Giải mã lệnh và phát tín hiệu điều khiển đến các thành phần khác, Các thanh ghi, Thanh ghi chung R0 R12 Sử dụng cho các phép tính và lưu trữ tạm thời dữ liệu, Thanh ghi đặc biệt, Bộ đếm chương trình PC Chỉ địa chỉ lệnh tiếp theo, Con trỏ ngăn xếp SP Quản lý ngăn xếp, Thanh ghi liên kết LR Lưu địa chỉ	và ngắt lồng nhau, Khối điều khiển hệ thống SCB, Quản lý các tính năng hệ thống như reset toàn bộ hệ thống và quản lý ngoại lệ, APB Bus ngoại vi nâng cao Kết nối với các thiết bị ngoại vi tốc độ thấp như UART SPI I2C 4 Bộ nhớ và Bus: Bản đồ bộ nhớ, Bộ nhớ Flash Lưu trữ chương trình cố định, SRAM Lưu trữ dữ liệu tạm thời trong quá trình thực thi chương trình, Kiến trúc Bus, AHB Bus hiệu suất cao nâng cao Kết nối lõi với các bộ nhớ và thiết bị ngoại vi tốc độ cao 5 Các chức năng hỗ trợ: Bộ đếm thời gian SysTick, Bộ đếm thời gian hệ thống cho các tác vụ định kỳ, Giám sát Digital, Giám sát và đảm bảo hệ thống không bị treo tự động reset khi cần thiết, Hỗ trợ gỡ lỗi, Cung cấp giao diện để lập trình viên gỡ lỗi và theo dõi hoạt động của vi xử lý
---	---

quay lại sau các lệnh gọi hàm, Thanh ghi trạng thái chương trình PSR Chứa các cờ trạng thái như N Negative Z Zero C Carry V Overflow 3 Hệ thống ngắt và xử lý ngoại lệ: Bộ điều khiển ngắt vector lồng nhau NVIC, Quản lý và ưu tiên các ngắt, Hỗ trợ xử lý ngắt ưu tiên cao	6 Đơn vị ngoại vi Peripherals: GPIO Đầu vào Đầu ra chung, Cấu hình các chân I O để kết nối với các thiết bị bên ngoài, Bộ đếm Thời gian, Sử dụng cho các ứng dụng đếm thời gian và tạo xung PWM, UART SPI I2C, Giao tiếp với các thiết bị ngoại vi như cảm biến màn hình và bộ nhớ ngoài
--	---

Flow Chart:

1. bắt đầu 2. khởi tạo hệ thống và các ngoại vi gọi hàm hal_init() cấu hình clock hệ thống với systemclock_config() khởi tạo gpio, uart1, timer2, adc1 qua các hàm: mx_gpio_init() mx_usart1_uart_init() mx_tim2_init() mx_adc1_init() khởi tạo lcd với lcd_init() bắt đầu timer interrupt với hal_tim_base_start_it(&tim2) 3. chờ nhấn nút start hệ thống chờ ngắt ngoài từ nút start (pa0) khi nút start được nhấn: gọi hàm startsystem() đặt biến system_running = 1 gửi thông báo "system started" qua uart 4. vòng lặp chính kiểm tra system_running nếu system_running == 1, thực hiện: đọc giá trị nhiệt độ từ cảm biến lm35 qua hàm read_temperature() kiểm tra sự thay đổi nhiệt độ nếu nhiệt độ thay đổi: kiểm tra nhiệt độ < 70°C nếu đúng: gửi giá trị nhiệt độ qua uart hiển thị nhiệt độ trên lcd tắt đèn led cảnh báo quá nhiệt (gpio pin 15) nếu sai (nhiệt độ ≥ 70°C): gửi thông báo "nhiệt độ cao" qua uart hiển thị "cao" trên lcd bật đèn led cảnh báo quá nhiệt (gpio pin 15) cập nhật biến previous_temp với nhiệt độ hiện tại delay 500ms (đảm bảo chu kỳ 0.5 giây)	5. xử lý ngắt uart (usart1_irqhandler) khi nhận dữ liệu từ uart: đọc ký tự nhận được (rx_char) phân tích ký tự nhận được: nếu là '1' đến '8': bật đèn led tương ứng (gpio pins 0-7) qua hàm led_control() nếu là '0': tắt tất cả đèn led nếu là 's': gửi trạng thái các đèn led qua uart với hàm send_led_state() nếu là 't': gửi giá trị nhiệt độ hiện tại qua uart với hàm send_temperature() nếu là 'a': đặt biến save = 1 (bắt đầu chế độ tự động) nếu là 'o': đặt biến save = 0 (tắt chế độ tự động) tắt tất cả đèn led 6. xử lý ngắt timer (tim2_irqhandler) mỗi 0.5 giây (theo cấu hình timer): đổi trạng thái đèn led "run" (gpio pin 11) để chỉ thị hệ thống đang chạy kiểm tra save != 0 nếu đúng: đổi trạng thái tất cả đèn led (gpio pins 0-7) để tạo hiệu ứng nhấp nháy 7. xử lý ngắt nút stop hệ thống chờ ngắt ngoài từ nút stop (pa1) khi nút stop được nhấn: gọi hàm stopsystem() đặt biến system_running = 0 gửi thông báo "system stopped" qua uart 8. kết thúc
---	--